C-Governed CLI Agent Mesh v0.1.1-hygiene

Executable worker governance for persistent c entities

Kotov Ivan
Bruxelles, 2026

Status	Draft protocol pack v0.1 with v0.1.1 hygiene synchronization
Date	2026-05-17
Package	C-Governed CLI Agent Mesh
Layer	c = a + b / SER / L4 / Agent Governance / CLI Worker Mesh / Defensive Adaptation / Witness
Document class	README / package landing page / release-boundary note
Assertion class	C-A10 package-control artifact
Primary subject	persistent c entities such as Ester and Liya

Contents

Status: Draft protocol pack v0.1 with v0.1.1 hygiene synchronization
Date: 2026-05-17
Package: C-Governed CLI Agent Mesh
Layer: `c = a + b` / SER / L4 / Agent Governance / CLI Worker Mesh / Defensive Adaptation / Witness
Document class: README / package landing page / release-boundary note
Assertion class: `C-A10` package-control artifact
Primary subject: persistent `c` entities such as Ester and Liya
Primary worker class: local, cloud, hybrid, portable, and quorum-participating CLI agents operating as bounded executable workers
Release boundary: draft protocol pack; not legal advice, not product certification, not deployment certification, not offensive security guidance.

---

0. Current package status

```
Architecture draft-complete: yes
v0.1.1 hygiene synchronization: completed at document level
Release-complete: not yet
Implementation-ready: not yet
Conformance-supported: not yet
Public-release-ready: not yet
Codex-ready for hygiene/schemas: yes, after human review of this synchronized set
```

This README supersedes the earlier v0.1 landing-page status language where it described missing README, release notes, contradiction register, public redaction, JSON schema extraction plan, fixture pack, registry, local checker, release surface, raw evidence sidecar, AB/gate profile, schema object registry, and semantic validator profile as absent.

Those companion documents now exist as draft package artifacts. That does **not** mean the package has executed conformance tests or extracted schemas.

Correct current claim:

```
Draft-complete governance package with v0.1.1 hygiene-synchronized package-control files;
ready for controlled schema-extraction and validation handoff.
```

Forbidden current claim:

```
release-complete
implementation-executed
conformance-supported
public-release-ready
certified safe
provider-certified
legally reviewed
```

---

1. Executive summary

C-Governed CLI Agent Mesh is a protocol pack for using local, cloud, hybrid, and portable CLI agents as bounded executable workers under the governance of a persistent `c`.

The package exists because CLI/cloud agents are no longer passive text helpers. They can inspect repositories, edit files, run commands, generate diffs, prepare releases, process logs, coordinate with other agents, and influence operational state.

That makes them useful.

It also makes them dangerous if they are allowed to become hidden authority.

Core sentence:

```
CLI agents are hands.
Not will.
Not memory.
Not sovereignty.
Not judge.
```

The package allows agents to:

```
inspect
propose
simulate
patch
test
compare
report
```

The package prohibits agents from:

```
self-authorizing
self-approving
writing core memory directly
mutating identity or continuity core
bypassing witness
leaking secrets into cloud context
turning defensive emulation into retaliation
performing hack-back
creating malware behavior
acting outside authorized scope
```

Compact formula:

```
Many executable hands.
One governed continuity.
No silent autonomy.
```

---

2. Corpus bridge set

2.1 Explicit bridge: `c = a + b`

In `c = a + b`, CLI agents belong to `b`: procedures, models, runtimes, tools, repositories, shell workers, cloud agents, local validators, and execution infrastructure.

They are not `a`.

They are not `c`.

They may assist `c`; they must not become `c` authority.

2.2 Quiet bridge I: information theory

A worker mesh without schemas, registries, and semantic validators leaks ambiguity. Ambiguity becomes authority when executable agents are allowed to interpret missing fields as permission.

2.3 Quiet bridge II: cybernetics

A control system needs brakes, sensors, negative feedback, and a clear controller. Task contracts, permissions, witnesses, memory gates, and rollback profiles prevent the worker mesh from becoming an uncontrolled actuator layer.

2.4 Earth paragraph

A good workshop does not give every worker the master key, the electrical panel, the invoices, the crane remote, and the client list. A worker gets a job card, a tool, a zone, a deadline, and an inspection. CLI agents need the same: task contract, permission, sandbox, witness, review, and a stop rule.

---

3. Core doctrine

The protocol stack is built around these invariants:

```
Capability is not permission.
Registration is not task authorization.
Cloud context is not private by default.
Quorum is evidence, not sovereignty.
Witness records boundaries, not raw life.
Raw evidence is a restricted sidecar, not witness text.
Memory is not a log sink.
Experience is not a diff.
Immunity is not retaliation.
Incident response is not revenge.
Conformance is not a promise; it is a tested boundary.
```

```
Schema validity is necessary but not sufficient.
Semantic validation enforces the boundary.
```

Operational doctrine:

```
Task contract before material work.
Scoped permission before capability use.
Registry eligibility before task assignment.
Sandbox before write.
Witness before privileged transition.
Review before integration.
Memory gate before memory.
Release gate before publication.
Human gate before high-risk consequence.
Fail closed on ambiguity.
```

---

4. Canonical documents

File	Role
`README.md`	Human entry point, status boundary, safe reading path.
`C-Governed_CLI_Agent_Mesh_Protocol_v0_1.md`	Root protocol and doctrine.
`CLI_Agent_Package_Index_and_Reading_Order_v0_1.md`	Canonical package map, reading order, dependency graph, release boundary.
`CLI_Agent_GLOSSARY_v0_1.md`	Terminology, class prefixes, disambiguation.
`CLI_Agent_RELEASE_NOTES_v0_1.md`	Release summary and claim boundary.
`CLI_Agent_OPEN_ISSUES_v0_1.md`	Open issues, blockers, deferred work.
`CLI_Agent_Contradiction_Register_v0_1.md`	Contradictions, soft tensions, release-readiness risks.
`CLI_Agent_v0_1_1_Hygiene_Patch_Notes.md`	Hygiene patch notes and synchronization record.
`CLI_Agent_Release_and_Implementation_Readiness_Gate_v0_1.md`	Release / implementation / conformance claim gates.
`CLI_Agent_Task_Contract_Schema_v0_1.md`	Task contract envelope.
`CLI_Agent_Permission_and_Capability_Model_v0_1.md`	Capability, permission, trust, drift, revocation.
`CLI_Agent_Handshake_Profile_v0_1.md`	Agent admission, provenance, challenge, registration.
`CLI_Agent_AB_Mode_and_Gate_Semantics_Profile_v0_1.md`	A/B, dry-run/apply/enforce, confirm phrases, arm gates.
`CLI_Agent_Registry_Profile_v0_1.md`	Agent inventory, eligibility, capability snapshots, revocation.
`CLI_Agent_Sandbox_Worktree_Profile_v0_1.md`	Sandbox/worktree/branch/container execution boundary.
`CLI_Agent_Witness_Event_Profile_v0_1.md`	Witness events, chains, references, boundary records.

File	Role
`CLI_Agent_Raw_Evidence_Sidecar_Profile_v0_1.md`	Raw evidence sidecar, custody, retention, disclosure separation.
`CLI_Agent_Memory_Gate_Profile_v0_1.md`	Memory, experience, policy, immunity promotion gate.
`CLI_Agent_Rollback_and_Freeze_Profile_v0_1.md`	Hold/freeze/quarantine/revoke/rollback/recovery.
`CLI_Agent_Quorum_and_Review_Profile_v0_1.md`	Multi-agent review and non-sovereign quorum.
`CLI_Agent_Executor_Reviewer_Separation_v0_1.md`	No self-approval, role separation, conflict detection.
`CLI_Agent_Defensive_Emulation_Boundaries_v0_1.md`	Restricted defensive emulation / garage / canary / no retaliation.
`CLI_Agent_Incident_Response_Profile_v0_1.md`	Triage, preservation, containment, repair, lawful handoff.
`CLI_Agent_Secrets_and_Cloud_Data_Policy_v0_1.md`	Secrets, cloud context, data classes, exposure handling.
`CLI_Agent_Public_Redaction_Profile_v0_1.md`	Public/restricted split and redaction rules.
`CLI_Agent_Release_Public_Surface_Profile_v0_1.md`	GitHub/site/archive/release/public-surface discipline.
`CLI_Agent_JSON_Schema_Extraction_Plan_v0_1.md`	Schema extraction plan and implementation handoff.
`CLI_Agent_Schema_Object_Registry_v0_1.md`	Canonical object inventory and schema map.
`CLI_Agent_Semantic_Validator_Rules_v0_1.md`	Semantic validation rules beyond JSON Schema.
`CLI_Agent_Local_Checker_Profile_v0_1.md`	Deterministic local checker and package hygiene preflight.
`CLI_Agent_Conformance_Test_Matrix_v0_1.md`	Conformance levels, tests, evidence, anti-washing.
`CLI_Agent_Conformance_Fixture_Pack_v0_1.md`	Safe synthetic fixtures for conformance tests.

---

5. Reading order

1. `README.md`
2. `C-Governed_CLI_Agent_Mesh_Protocol_v0_1.md`
3. `CLI_Agent_Package_Index_and_Reading_Order_v0_1.md`
4. `CLI_Agent_GLOSSARY_v0_1.md`
5. `CLI_Agent_RELEASE_NOTES_v0_1.md`
6. `CLI_Agent_OPEN_ISSUES_v0_1.md`
7. `CLI_Agent_Contradiction_Register_v0_1.md`
8. `CLI_Agent_v0_1_1_Hygiene_Patch_Notes.md`
9. `CLI_Agent_Release_and_Implementation_Readiness_Gate_v0_1.md`
10. `CLI_Agent_Task_Contract_Schema_v0_1.md`
11. `CLI_Agent_Permission_and_Capability_Model_v0_1.md`

12. `CLI_Agent_Handshake_Profile_v0_1.md`
13. `CLI_Agent_AB_Mode_and_Gate_Semantics_Profile_v0_1.md`
14. `CLI_Agent_Registry_Profile_v0_1.md`
15. `CLI_Agent_Sandbox_Worktree_Profile_v0_1.md`
16. `CLI_Agent_Witness_Event_Profile_v0_1.md`
17. `CLI_Agent_Raw_Evidence_Sidecar_Profile_v0_1.md`
18. `CLI_Agent_Memory_Gate_Profile_v0_1.md`
19. `CLI_Agent_Rollback_and_Freeze_Profile_v0_1.md`
20. `CLI_Agent_Quorum_and_Review_Profile_v0_1.md`
21. `CLI_Agent_Executor_Reviewer_Separation_v0_1.md`
22. `CLI_Agent_Defensive_Emulation_Boundaries_v0_1.md`
23. `CLI_Agent_Incident_Response_Profile_v0_1.md`
24. `CLI_Agent_Secrets_and_Cloud_Data_Policy_v0_1.md`
25. `CLI_Agent_Public_Redaction_Profile_v0_1.md`
26. `CLI_Agent_Release_Public_Surface_Profile_v0_1.md`
27. `CLI_Agent_JSON_Schema_Extraction_Plan_v0_1.md`
28. `CLI_Agent_Schema_Object_Registry_v0_1.md`
29. `CLI_Agent_Semantic_Validator_Rules_v0_1.md`
30. `CLI_Agent_Local_Checker_Profile_v0_1.md`
31. `CLI_Agent_Conformance_Test_Matrix_v0_1.md`
32. `CLI_Agent_Conformance_Fixture_Pack_v0_1.md`

For a minimal conceptual path:

1. `README.md`
2. `C-Governed_CLI_Agent_Mesh_Protocol_v0_1.md`
3. `CLI_Agent_Task_Contract_Schema_v0_1.md`
4. `CLI_Agent_Permission_and_Capability_Model_v0_1.md`
5. `CLI_Agent_Handshake_Profile_v0_1.md`
6. `CLI_Agent_AB_Mode_and_Gate_Semantics_Profile_v0_1.md`
7. `CLI_Agent_Registry_Profile_v0_1.md`
8. `CLI_Agent_Sandbox_Worktree_Profile_v0_1.md`
9. `CLI_Agent_Witness_Event_Profile_v0_1.md`
10. `CLI_Agent_Raw_Evidence_Sidecar_Profile_v0_1.md`
11. `CLI_Agent_Executor_Reviewer_Separation_v0_1.md`
12. `CLI_Agent_Secrets_and_Cloud_Data_Policy_v0_1.md`
13. `CLI_Agent_Conformance_Test_Matrix_v0_1.md`
14. `CLI_Agent_Schema_Object_Registry_v0_1.md`

15. `CLI_Agent_Semantic_Validator_Rules_v0_1.md`

---

6. Operational flow

Normal task flow:

```
agent candidate
  -> registry lookup
  -> handshake
  -> capability snapshot
  -> trust / auto-connect ceiling
  -> task contract
  -> permission grant
  -> sandbox/worktree
  -> AB/gate evaluation
  -> execution
  -> witness
  -> local checker
  -> review/quorum
  -> c gate
  -> human gate if high-risk
  -> integration / memory gate / release / rollback / quarantine
```

Defensive / incident flow:

```
suspicious signal
  -> triage
  -> freeze or hold if needed
  -> preserve enough evidence
  -> raw evidence sidecar if restricted evidence exists
  -> contain locally
  -> sandbox replay / defensive emulation if justified
  -> repair in sandbox
  -> validate
  -> memory gate / immunity candidate
  -> witness
  -> lawful report or handoff if needed
```

No step authorizes retaliation.

---

7. Public / restricted status

Public-capable after review:

```
root protocol
README
package index
readiness gate
AB/gate profile
registry profile with synthetic examples only
task contract
permission model
```

```
handshake
sandbox/worktree
witness profile
raw evidence sidecar profile without real evidence
memory gate
rollback/freeze
quorum/review
executor/reviewer separation
public redaction
release surface
JSON schema extraction plan
schema object registry
semantic validator rules
local checker profile
conformance matrix
fixture pack
open issues
contradiction register
release notes
```

Restricted or redaction-required before public release:

```
defensive emulation boundaries
incident response profile
secrets and cloud data policy
real conformance outputs
real incident reports
real provider boundary records
real secret boundary records
real cloud exposure events
real memory-gate records involving private material
real raw evidence sidecars
real garage/canary/signature material
```

Public release must not include:

```
real secrets
real credentials
real cloud account refs
real local infrastructure paths
real incident evidence
real private memory
legal privileged material
operational canary values
exploit-like details
unredacted provider boundary records
```

---

8. Conformance status

Current status:

```
Conformance matrix: present
Conformance fixture pack: present as draft safe synthetic catalog
Schema object registry: present
Semantic validator rules: present as normative plan
Local checker profile: present
Extracted JSON schemas: not yet created
Reference validator: not yet implemented
```

```
Actual test runs: not yet executed
Evidence packets: not yet generated
Conformance-supported claim: forbidden
```

Correct current statement:

```
The package defines a conformance framework and safe fixture basis, but v0.1.1 has not yet
executed conformance tests.
```

---

9. Remaining blockers before release-complete

Release-complete still requires:

1. canonical filename inventory and duplicate-source resolution, especially sandbox/worktree variants;
2. public/restricted split applied to the actual repository package;
3. final repository placement and default-branch discoverability;
4. release manifest / artifact integrity records generated after Markdown freeze;
5. package-control files committed together, not scattered across side branches.

---

10. Remaining blockers before implementation-ready

Implementation-ready still requires:

1. P0 schema extraction task contract;
2. `SCHEMA_INDEX.json` target or draft;
3. semantic validator implementation plan bound to `CLI_Agent_Semantic_Validator_Rules_v0_1.md`;
4. fixture binding plan;
5. local checker run contract;
6. no stale package-control claims;
7. Codex handoff with `memory=off`, `auto_ingest=false`, no publication, no release tag.

Implementation-ready does not require conformance to have passed.

---

11. Release checklist

Before public or archival release:

- ☐ Confirm all canonical filenames and remove/supersede duplicate source variants.
- ☐ Confirm `CLI_Agent_Package_Index_and_Reading_Order_v0_1.md` exists in the package.
- ☐ Confirm README links every canonical file.
- ☐ Apply public/restricted split.
- ☐ Redact sensitive examples where needed.
- ☐ Ensure no operational offensive detail slipped into DEB/IRP/SCDP.
- ☐ Freeze Markdown.
- ☐ Generate PDFs only after Markdown freeze.
- ☐ Generate `SHA256SUMS` only after all final artifacts exist.
- ☐ Prepare release manifest.
- ☐ Tag release only after final review.

---

12. Closing rule

A persistent `c` can use many agents without becoming a swarm.

But only if the agents remain governed.

Final rule:

```
If a CLI agent can touch the world,
it must be governed before it touches memory,
reviewed before it touches release,
witnessed before it touches privilege,
and stopped before it crosses a red line.
```